

UNIVERSITY OF WAH(UOW)

**DEPARTMENT OF COMPUTER
SCIENCE**



COURSE:

Information & Security lab - 251L

SUBMITTED TO:

Ms. Wasifa Kanwal

SUBMITTED BY:

Muhammad Talha

Section

CS.3ndB

Registration NO:

UW-24-CS-BS-101

Lab Task # 03

Task # 01

```
email = {
    "from": "support@bank-secure-login.com",
    "to": "student@uni.edu",
    "subject": "URGENT: verify Acoount now",
    "body": "Dear user, click http://bank-secure-login.com/verify and enter your credential to avoid s
}

print("---Received Email---")

for k,v in email.items():
    print(f"{k}, {v}")
warning = []
if "urgent" in email["subject"].lower():
    warning.append("Urgent language -> social engineering red flag")
if "http://" in email["body"] and "https://" not in email["body"]:
    warning.append("Contains non-HTTPS link")
if email["from"].split("@")[-1].count("-") > 0 or len(email["from"].split("@")[-1].split(".")) < 2 :
    warning.append("Suspicious Sender Domain")
print("\nChecks/Warning: ")
if warning:
    for w in warning:
        print(f" - {w}")
else:
    print("- No obvious red flags (still be careful)")
```

Output

```
---Received Email---
from, support@bank-secure-login.com
to, student@uni.edu
subject, URGENT: verify Acoount now
body, Dear user, click http://bank-secure-login.com/verify and enter

Checks/Warning:
- Urgent language -> social engineering red flag
- Contains non-HTTPS link
- Suspicious Sender Domain
```

Code

```
email = {
    "from": "support@bank-secure-login.com",
    "to": "student@uni.edu",
    "subject": "URGENT: verify Acoount now",
    "body": "Dear user, click http://bank-secure-login.com/verify and enter your credential to
avoid suspension."
}
print("---Received Email---")
for k,v in email.items():
    print(f"{k}, {v}")
warning = []
if "urgent" in email["subject"].lower():
    warning.append("Urgent language -> social engineering red flag")
```

```

if "http://" in email["body"] and "https://" not in email["body"]:
    warning.append("Contains non-HTTPS link")
if email["from"].split("@")[-1].count("-") > 0 or len(email["from"].split("@")[-1].split("."))
< 2 :
    warning.append("Suspicious Sender Domain")
print("\nChecks/Warning: ")
if warning:
    for w in warning:
        print(f" - {w}")
else:
    print("- No obvious red flags (still be carefull)")

```

Task # 02

```

import time
from queue import Queue
💡
SERVICE_CAPACITY = 5
request = Queue()

for i in range(8):
    request.put(f"req_{i}")

processed = 0
print("Service starting capacity: ",SERVICE_CAPACITY)
while not request.empty():
    if processed >= SERVICE_CAPACITY:
        print("Service overloading: response time incresing...")
        time.sleep(1)
    req = request.get()
    print("Processing",req)
    processed += 1
    time.sleep(0.1)

```

Output

```

Service starting capacity: 5
Processing req_0
Processing req_1
Processing req_2
Processing req_3
Processing req_4
Service overloading: response time incresing...
Processing req_5
Service overloading: response time incresing...
Processing req_6
Service overloading: response time incresing...
Processing req_7

```

Code

```

import time
from queue import Queue

SERVICE_CAPACITY = 5
request = Queue()

for i in range(8):
    request.put(f"req_{i}")

processed = 0
print("Service starting capacity: ",SERVICE_CAPACITY)
while not request.empty():
    if processed >= SERVICE_CAPACITY:
        print("Service overloading: response time incresing...")
        time.sleep(1)
    req = request.get()
    print("Processing",req)
    processed += 1
    time.sleep(0.1)

```

Task # 03

```

users = {"student": "letmein123"}

def authentiaction(user,password):
    return users.get(user) == password

wordlist = ["123","password","letmein","letmein123"]

for guess in wordlist:
    print("Trying: ",guess)
    if authentiaction("student",guess):
        print("Password found: ",guess)
        break
else:
    print("Password not found in the tested wordlist")

```

Output

```

Trying: 123
Trying: password
Trying: letmein
Trying: letmein123
Password found: letmein123

```

Code

```

users = {"student": "letmein123"}

def authentiaction(user,password):
    return users.get(user) == password

```

```
wordlist = ["123","password","letmein","letmein123"]

for guess in wordlist:
    print("Trying: ",guess)
    if authentication("student",guess):
        print("Password found: ",guess)
        break
else:
    print("Password not found in the tested wordlist")
```

Task # 04

```
def sender(msg):
    print("[Sender] Sending: ",msg)
    return msg

def network_path(message):
    print("[Attacker] Intercepted: ",message)
    return message

def receiver(msg):
    print("[Receiver] Received: ",msg)

if __name__ == "__main__":
    m = sender("Sensitive : Password:abc123")
    m2 = network_path(m)
    receiver(m2)
```

Output

```
[Sender] Sending: Sensitive : Password:abc123
[Attacker] Intercepted: Sensitive : Password:abc123
[Receiver] Received: Sensitive : Password:abc123
```

Code

```
def sender(msg):
    print("[Sender] Sending: ",msg)
    return msg

def network_path(message):
    print("[Attacker] Intercepted: ",message)
    return message

def receiver(msg):
    print("[Receiver] Received: ",msg)

if __name__ == "__main__":
    m = sender("Sensitive : Password:abc123")
    m2 = network_path(m)
    receiver(m2)
```

Task # 05

```
print("Smart Email Security Filter")
suspicious_words = ["urgent", "verify", "click here", "account"]
low_risk_count = 0
medium_risk_count = 0
high_risk_count = 0
for i in range(1, 4):
    print(f"Enter details for Email {i}")
    subject = input("Enter the subject: ").lower()
    body = input("Enter the body: ").lower()
    full_text = subject + " " + body
    found_words = [word for word in suspicious_words if word in full_text]
    red_flags = len(found_words)
    if red_flags == 0:
        risk = "Low Risk"
        low_risk_count += 1
    elif red_flags == 1:
        risk = "Medium Risk"
        medium_risk_count += 1
    else:
        risk = "High Risk"
        high_risk_count += 1
    print(f"Email {i} Analysis:")
    print(f" - Suspicious words found: {found_words if found_words else 'None'}")
    print(f" - Total red flags: {red_flags}")
    print(f" - Final Risk Level: {risk}")
print("\nOverall Summary")
print(f"Total Emails Analyzed: 3")
print(f"Low Risk Emails: {low_risk_count}")
print(f"Medium Risk Emails: {medium_risk_count}")
print(f"High Risk Emails: {high_risk_count}")
```

Output

```
Smart Email Security Filter
Enter details for Email 1
Enter the subject: Student attendance!
Enter the body: Enter your attendance please!
Email 1 Analysis:
- Suspicious words found: None
- Total red flags: 0
- Final Risk Level: Low Risk
Enter details for Email 2
Enter the subject: verify email
Enter the body: Hello sir please verrify your email.
Email 2 Analysis:
- Suspicious words found: ['verify']
- Total red flags: 1
- Final Risk Level: Medium Risk
Enter details for Email 3
Enter the subject: Important Node
Enter the body: Click here to receive reward.
Email 3 Analysis:
- Suspicious words found: ['click here']
- Total red flags: 1
- Final Risk Level: Medium Risk

Overall Summary
Total Emails Analyzed: 3
Low Risk Emails: 1
Medium Risk Emails: 2
High Risk Emails: 0
```

Code

```
print("Smart Email Security Filter")
suspicious_words = ["urgent", "verify", "click here", "account"]
low_risk_count = 0
medium_risk_count = 0
high_risk_count = 0
for i in range(1, 4):
    print(f"Enter details for Email {i}")
    subject = input("Enter the subject: ").lower()
    body = input("Enter the body: ").lower()
    full_text = subject + " " + body
    found_words = [word for word in suspicious_words if word in full_text]
    red_flags = len(found_words)
    if red_flags == 0:
        risk = "Low Risk"
        low_risk_count += 1
    elif red_flags == 1:
        risk = "Medium Risk"
        medium_risk_count += 1
    else:
        risk = "High Risk"
        high_risk_count += 1
    print(f"Email {i} Analysis:")
```

```
print(f" - Suspicious words found: {found_words if found_words else 'None'}")
print(f" - Total red flags: {red_flags}")
print(f" - Final Risk Level: {risk}")
print("\nOverall Summary")
print(f"Total Emails Analyzed: 3")
print(f"Low Risk Emails: {low_risk_count}")
print(f"Medium Risk Emails: {medium_risk_count}")
print(f"High Risk Emails: {high_risk_count}")
```

Task # 06

```
def sender(msg):
    checksum = sum(ord(c) for c in msg)
    packet = f"{msg}|{checksum}"
    print("Sender -> packet:", packet)
    return packet

def network(packet, tamper=False):
    msg, checksum_str = packet.rsplit("|", 1)
    if tamper:
        if "c" in msg:
            tampered_msg = msg.replace("c", "", 1)
        else:
            tampered_msg = msg[:-1] if len(msg) > 0 else msg
        tampered_packet = f"{tampered_msg}|{checksum_str}"
        print("Network (attacker) -> packet:", tampered_packet)
        return tampered_packet
    else:
        print("Network -> packet forwarded:", packet)
        return packet

def receiver(packet):
    if "|" not in packet:
        print("Receiver -> Invalid packet format")
        return
    msg, checksum_str = packet.rsplit("|", 1)
    try:
        received_checksum = int(checksum_str)
    except ValueError:
        print("Receiver -> Invalid checksum field")
        return
```



```

    computed_checksum = sum(ord(c) for c in msg)
    print("Receiver -> received msg:", msg)
    print("Receiver -> received checksum:", received_checksum)
    print("Receiver -> computed checksum:", computed_checksum)
    if computed_checksum == received_checksum:
        print("Receiver -> OK")
    else:
        print("Receiver -> Tampered!")

print(" Msg 1: No Tampering ")
p = sender("hello_abc123")
p_fwd = network(p, tamper=False)
receiver(p_fwd)

print(" Msg 2: With Tampering ")
p2 = sender("hello_abc123")
p2_tampered = network(p2, tamper=True)
receiver(p2_tampered)

```

Output

```

Msg 1: No Tampering
Sender -> packet: hello_abc123|1071
Network -> packet forwarded: hello_abc123|1071
Receiver -> received msg: hello_abc123
Receiver -> received checksum: 1071
Receiver -> computed checksum: 1071
Receiver -> OK
Msg 2: With Tampering
Sender -> packet: hello_abc123|1071
Network (attacker) -> packet: hello_ab123|1071
Receiver -> received msg: hello_ab123
Receiver -> received checksum: 1071
Receiver -> computed checksum: 972
Receiver -> Tampered!

```

Code

```

def sender(msg):
    checksum = sum(ord(c) for c in msg)
    packet = f"{msg}|{checksum}"
    print("Sender -> packet:", packet)
    return packet

def network(packet, tamper=False):
    msg, checksum_str = packet.rsplit("|", 1)
    if tamper:
        if "c" in msg:
            tampered_msg = msg.replace("c", "", 1)
        else:
            tampered_msg = msg[:-1] if len(msg) > 0 else msg
        tampered_packet = f"{tampered_msg}|{checksum_str}"
        print("Network (attacker) -> packet:", tampered_packet)
        return tampered_packet
    else:

```

```

        print("Network -> packet forwarded:", packet)
        return packet
def receiver(packet):
    if "|" not in packet:
        print("Receiver -> Invalid packet format")
        return
    msg, checksum_str = packet.rsplit("|", 1)
    try:
        received_checksum = int(checksum_str)
    except ValueError:
        print("Receiver -> Invalid checksum field")
        return
    computed_checksum = sum(ord(c) for c in msg)
    print("Receiver -> received msg:", msg)
    print("Receiver -> received checksum:", received_checksum)
    print("Receiver -> computed checksum:", computed_checksum)
    if computed_checksum == received_checksum:
        print("Receiver -> OK")
    else:
        print("Receiver -> Tampered!")

print(" Msg 1: No Tampering ")
p = sender("hello_abc123")
p_fwd = network(p, tamper=False)
receiver(p_fwd)

print(" Msg 2: With Tampering ")
p2 = sender("hello_abc123")
p2_tampered = network(p2, tamper=True)
receiver(p2_tampered)

```

Task # 06

```

def process_requests(requests, capacity, per_source_limit):
    processed_count = 0
    source_count = {}
    blocked_sources = set()

    for req_id, source in requests:
        if source in blocked_sources:
            print(f"Blocked {req_id} from {source}")
            continue
        if processed_count >= capacity:
            print("Server overloaded!")
            break

        source_count[source] = source_count.get(source, 0) + 1
        if source_count[source] > per_source_limit:
            blocked_sources.add(source)
            print(f"Blocked {req_id} from {source}")
            continue

```

```

    print(f"Processing {req_id} from {source}")
    processed_count += 1

print("\n--- Summary ---")
print(f"Total processed: {processed_count}")
print(f"Blocked sources: {blocked_sources if blocked_sources else 'None'}")

requests = [
    ("R1", "10.0.0.1"),
    ("R2", "10.0.0.2"),
    ("R3", "10.0.0.1"),
    ("R4", "10.0.0.1"),
    ("R5", "10.0.0.3"),
    ("R6", "10.0.0.1"),
    ("R7", "10.0.0.2"),
    ("R8", "10.0.0.4"),
]

process_requests(requests, capacity=6, per_source_limit=2)

```

Output

```

Processing R1 from 10.0.0.1
Processing R2 from 10.0.0.2
Processing R3 from 10.0.0.1
Blocked R4 from 10.0.0.1
Processing R5 from 10.0.0.3
Blocked R6 from 10.0.0.1
Processing R7 from 10.0.0.2
Processing R8 from 10.0.0.4

--- Summary ---
Total processed: 6
Blocked sources: {'10.0.0.1'}

```

Code

```

def process_requests(requests, capacity, per_source_limit):
    processed_count = 0
    source_count = {}
    blocked_sources = set()

    for req_id, source in requests:
        if source in blocked_sources:
            print(f"Blocked {req_id} from {source}")
            continue
        if processed_count >= capacity:
            print("Server overloaded!")
            break

        source_count[source] = source_count.get(source, 0) + 1
        if source_count[source] > per_source_limit:

```

```
        blocked_sources.add(source)
        print(f"Blocked {req_id} from {source}")
        continue

    print(f"Processing {req_id} from {source}")
    processed_count += 1

print("\n--- Summary ---")
print(f"Total processed: {processed_count}")
print(f"Blocked sources: {blocked_sources if blocked_sources else 'None'}")

requests = [
    ("R1", "10.0.0.1"),
    ("R2", "10.0.0.2"),
    ("R3", "10.0.0.1"),
    ("R4", "10.0.0.1"),
    ("R5", "10.0.0.3"),
    ("R6", "10.0.0.1"),
    ("R7", "10.0.0.2"),
    ("R8", "10.0.0.4"),
]
process_requests(requests, capacity=6, per_source_limit=2)
```